CPS572 Final Project

Ahmad Choudhary, Kevin Han, Shashwat Rao, Hemasai Suhas Kurapati

1 Methodology

1.1 Dataset Selection

The initial step in our pipeline was to curate relevant datasets for fine-tuning. We started with those recommended by the assignment, which came from three primary sources: `openai/gsm8k` for math (7,473 samples), `allenai/tulu-3-sft-mixture` for instruction following (approximately 939K samples), and `nvidia/OpenCodeInstruct` for code (approximately 5M samples). Our initial strategy was to train on all three datasets equally to improve performance across all evaluation benchmarks. However, we observed a bottleneck with the `tulu-3-sft-mixture` dataset, particularly in improving IFEval scores.

To address this, we explored alternative datasets for instruction following and identified `allenai/tulu-3-sft-personas-instruction-following`. Replacing the mixture dataset with this personas dataset, while keeping all hyperparameters and training configurations fixed. This change resulted in a significant improvement in IFEval performance, increasing from 0.28 to 0.49 on the 3B model. Although substantially smaller, the personas dataset is more specialized for instruction-following tasks, whereas the mixture dataset is broader with noise less relevant to the tasks.

Math posed a similar problem. To mitigate, we evaluated alternative datasets and selected `metamath/MetaMathQA`. This dataset yielded a modest improvement of approximately 2–3% on math benchmarks. While smaller in magnitude, these gains became more meaningful at larger training scales. The final training configuration used approximately 12,000 datapoints, exceeding the size of the original GSM8K training split. Additionally, MetaMathQA has been explicitly shown to improve GSM8K performance, making it well-suited for our use case.

Improving coding performance proved more challenging. Experiments with datasets such as `google-research-datasets/mbpp` resulted in decreased performance. Instead, we observed that HumanEval scores improved indirectly through better instruction-following and math capabilities, as well as through increased exposure to samples from `OpenCodeInstruct`. Given that large language models are already heavily pretrained on code, additional fine-tuning on code-specific datasets appears to yield diminishing returns compared to more underrepresented domains such as math and instruction following.

1.2 Dataset Curation

We initially allocated data equally across all tasks, but this quickly proved suboptimal. Since the model already demonstrated relatively strong performance on coding tasks in the baseline, we inferred emphasis on code data would yield diminishing returns. Given the compute budget, it was more effective to prioritize datasets in instruction following and mathematical reasoning.

To determine the appropriate allocation between these two domains, we conducted experiments varying the proportion of instruction-following and math data. We found that increasing the amount of instruction-following data (Tulu) consistently produced improvements not only in IFEval but also modest gains in other benchmarks. In contrast, scaling the math dataset yielded more isolated improvements. Based on these observations, the final dataset ratio was set to 15,000 samples from Tulu, 12,000 from math (MetaMathQA), and 5,000 from OpenCodeInstruct.

Training was conducted for 6,400 steps with a batch size of 10, corresponding to approximately

64,000 total training samples. This configuration was chosen to provide roughly two epochs over the combined dataset. Empirically, we found that two epochs provided a favorable balance between underfitting and overfitting. A single epoch with more unique data resulted in insufficient adaptation, while increasing the number of epochs on a smaller dataset led to overfitting and reduced generalization across benchmarks. We did not utilize the full extent of any dataset

1.3 Supervised Fine-Tuning (SFT)

Using the dataset composition and ratios described above, we performed supervised fine-tuning on the 8B parameter model using Low-Rank Adaptation (LoRA), which decomposes matrix weight updates as the product of two low rank matrices, saving memory. The LoRA configuration used a rank of 32 and a learning rate of 5×10^{-5} with a warmup and a cosine schedule. Training was conducted over several hours under the step and batch size settings discussed in this report.

The resulting model achieved scores of 73.2/69.1/48.1 on IFEval, math, and HumanEval, respectively. These results represent a substantial improvement over baseline performance and exceed the benchmark by a significant margin.

To ensure the validity of these results, we performed a contamination check in the provided file: `check_contamination.py`. An exact match search was conducted between the training data and evaluation sets, and no overlap was found. This confirms that the reported performance in our final submission is not influenced by data leakage. Preventing data contamination was an ongoing process, and we discuss a specific contamination case from our earlier experimentation in §3.2.

1.4 Reinforcement Learning (RL) Fine-Tuning

We further explored reinforcement learning–based methods introduced in class to improve performance. Specifically, we evaluated Direct Preference Optimization (DPO) and Group Relative Policy Optimization (GRPO), both implemented using Tinker's provided training pipelines and associated datasets. In all cases, these methods were applied on top of the previously trained SFT model.

For DPO, we experimented with several configurations and datasets, ultimately selecting the `ultrafeedback` dataset as it provided the most consistent improvements. The final configuration used a batch size of 32, 200 training steps, and a learning rate of 1×10^{-5}. This setup yielded scores of 72.8/71/50 on IFEval, math, and HumanEval, respectively. Compared to SFT alone, DPO introduced a tradeoff: while math and coding performance improved, instruction-following performance saw a slight decrease.

To further explore optimization strategies, we applied GRPO using Tinker's default configuration, which utilizes the GSM8K training split. Given the dataset, we expected improvements primarily in mathematical reasoning. Training was conducted for one epoch over the dataset with a batch size of 32, 233 steps, and a group size of 8. We chose this set of hyperparameters based on automated and heuristic experimentation, noting the points beyond which performance degraded. This approach resulted in scores of 74.3/78/52.4, outperforming both the SFT and DPO models. As anticipated, math performance saw the largest gain, but instruction-following and coding performance also improved by approximately 2%, indicating that GRPO provided more balanced overall enhancements.

2 Results

Table 1 summarizes performance across all evaluation settings for the baseline (dummy), SFT, SFT+DPO, and SFT+GRPO models.

Model	IFEval	GSM8K (Math)	HumanEval	Average
Base (3B)	0.2642	0.1979	0.3049	0.2557
SFT (8B)	0.7321	0.6914	0.4817	0.6351
SFT + DPO (8B)	0.7287	0.7104	0.5000	0.6464
SFT + GRPO (8B)	**0.7439**	**0.7809**	**0.5244**	**0.6831**

Table 1: Final evaluation accuracy across models. Best results are highlighted in bold.

The SFT model provides a substantial improvement over the baseline across all benchmarks, confirming the effectiveness of dataset curation and proportional training. Applying DPO on top of SFT introduces modest gains in math and code performance, though with a slight decrease in IFEval. In contrast, GRPO yields the strongest overall performance, achieving the highest scores across all three benchmarks. As expected, GRPO leads to the largest improvement in math due to its reliance on GSM8K during training, while also providing consistent gains in instruction following and coding.

Overall, these results indicate that while SFT accounts for the majority of performance gains, reinforcement learning–based methods, particularly GRPO, provide additional improvements and help achieve better cross-benchmark generalization.

3 Analysis

3.1 What Worked

SFT with curated datasets provided a substantial improvement over baseline performance across all benchmarks. The personas dataset for instruction following resulted in a significant improvement in IFEval performance compared to the Tulu mixture dataset, likely because it is more specialized and contains less noise relative to the target evaluation.

Increasing the amount of instruction-following data consistently produced improvements not only in IFEval but also modest gains in other benchmarks. While smaller in magnitude, improvements from MetaMathQA became more meaningful at larger training scales, and the dataset is well-suited for improving GSM8K performance.

RL based algorithms such as GRPO further improved performance across all benchmarks and provided the strongest overall results. When considering RL algorithms, we chose GRPO as generations could be done in parallel and doesn't require a value network for advantage estimate (compared to PPO), making it an easier to use method not reliant on many other components. Specifically, we utilized GRPO on the final answers of the GSM8K dataset since this was provided in the Tinker API. As a result GRPO led to the largest improvement in math while also improving instruction-following and coding performance by approximately 2%.

3.2 What Did Not Work

Curriculum Learning We experimented with curriculum learning, enabling the model to learn easier tasks (Instruction Following) before more difficult ones (Code). We did this by adjusting the data mixing for the train sample throughout the training process (e.g. decreasing the proportion of IF data, increasing the proportion of Code data). Implementing this feature significantly worsened IF performance, likely because too small of a proportion of the training data was IF-focused towards the end of training. We also thought that this poor performance could be due to significant differences in the format and necessary steps involved in completing these tasks, which makes the thesis of curriculum learning invalid.

Other GRPO Implementations We implemented a custom GRPO variant in which a stronger external model (OSS-120B) was used to evaluate generated outputs and provide an additional reward for answers that did not align with the ground truth, which was given the full reward. However, this approach was ultimately deprecated due to unstable training dynamics and lack of consistent improvements. One likely issue is reward misalignment: the external judge model may not perfectly reflect the evaluation criteria of the target benchmarks, leading to noisy or biased reward signals. Additionally, using a learned model as a verifier introduces variance and inconsistency compared to deterministic checks (e.g., exact match or unit tests), which can destabilize policy updates in GRPO.

For dataset selection: We initially included `ise-uiuc/Magicoder-Evol-Instruct-110K` as a code source. It bumped up performance in a teammate's testing configuration, but our deep audits found that contained Evol-Instruct paraphrases of 22% of HumanEval problems with distinctive examples preserved verbatim (see §3.4). We removed it and iterated to improve after this −4.88 pp HumanEval penalty.

Another notable challenge was the MBPP experiment described in §1.1 (Dataset selection, last paragraph). We saw that direct supervised fine-tuning on the MBPP (Mostly Basic Python Problems) sanitized train split consistently lowered HumanEval, even at small mix proportions. Our evaluation is that MBPP's prompt style (generally single line natural language descriptions of small standalone functions evaluated by `assert` statements) was sufficiently different from HumanEval's docstring-driven, example based format. This caused the SFT to pull the model's code distribution toward MBPP's shape and away from HumanEval's. We treated this as a reminder that not every code dataset transfers across code benchmarks, and kept `nvidia/OpenCodeInstruct` as our sole code source for the final mix.

4 Extensions

We explored several extensions beyond standard supervised fine-tuning to further improve performance across benchmarks:

1. **Performance Based Dataset Curation** Using our results from initial runs on the eval data, we adjusted our training runs to use more from the tasks our model performed worse on (e.g., personas-based instruction-following and MetaMathQA), which led to stronger targeted improvements. Here are the two datasets we used that were not in the recommened:

   `https://huggingface.co/datasets/allenai/tulu-3-sft-personas-instruction-following`

   `https://huggingface.co/datasets/meta-math/MetaMathQA`
2. **Hyperparameter Tuning** Additionally, we did multiple runs with varying batch sizes, learning rate schedules, and steps to ensure that our runs were using optimal hyper-parameters. Two epochs was found to be the most optimal in data representation and a batch size of 10 was kept reletively small so the model can learn as much as possible from the limited data.
3. **Direct Preference Optimization (DPO)** We applied the DPO RL algorithm on the `ultrafeedback` dataset, which yielded moderate gains, particularly in math and code tests, with minor tradeoffs in instruction following.
4. **Tinker Group Relative Policy Optimization (GRPO)** Using the default GSM8K-based GRPO pipeline in the Tinker API gave us improvements on the math test result, while also improving performance across other benchmarks. In long runs, like the assignment stated, performance began to diminish in IFEval and Human Eval due to the math focus so we found 233 steps to be the max in improving the benchmark scores.

LLM Usage

We used AI-based tools including **Claude Code** (Anthropic CLI agent with Opus 4.7), **OpenAI ChatGPT**, and **Gemini** throughout this project. We used these tools to accelerate development, explore ideas, and assist with implementation. We made all final decisions based on manual validation, training results, and benchmark evaluations.

- **Code writing and refactoring** We used LLMs to draft and refactor several components, contamination audit scripts, dataset pre-filtering logic, and SFT/RL configuration code. We reviewed, tested, and modified all generated code before integrating it into our system.
- **Debugging and systems integration** We used LLMs to diagnose and resolve development issues, including interpreting Tinker API errors and improving performance of the audit pipeline through parallelization.
- **Experimentation and idea generation** We used LLMs to propose candidate approaches, including multi-task RL strategies, BoN distillation, KodCode integration, and dataset filtering methods. We evaluated these suggestions through experiments and selected approaches based on observed performance rather than LLM recommendations alone.

We applied human judgment on top of all LLM outputs. We analyzed training logs, compared evaluation metrics across runs, and selected approaches based on the specific bottlenecks observed during experimentation. During debugging, we relied primarily on manual inspection and testing.

We identified multiple incorrect or misleading LLM outputs during the project. These included references to nonexistent datasets, unsupported hyperparameter configurations, and inaccurate claims about expected performance. We treated all LLM outputs as unverified suggestions and validated them through independent analysis.

One notable example involved a $\sim$22% HumanEval contamination signal in Magicoder-Evol. Our manual audit identified this issue even after the LLM initially attributed it to false positives. Only after tracing specific examples did the LLM acknowledge the contamination. This reinforced our approach of verifying all LLM-generated claims.

Tinker Feedback

Challenges

- **RL Pipeline:** We found the RL pipeline abstractions difficult to understand, particularly the interaction between `ProblemEnv`, `EnvGroupBuilder`, `RLDataset`, and `RLDatasetBuilder`.This sometimes increased the time between iterations during initial development, simply due to confusion.
- **Dataset exhaustion:** `InterleavedRLDatasetBuilder` determines the total number of training steps based on the smallest dataset. This caused runs to terminate earlier than expected. For example, one run stopped at step 7 despite a configured `max_steps=60` because a filtered dataset contained only ∼120 samples. This behavior was not clearly documented and required significant debugging.
- **Tinker abstractions:** Tinker abstracts away many parts of the training process, especially adjustments to model internals such as architectural changes. While it keeps things simple, it means various changed we suggested weren't actually implementable (Though this may be more of a Meta LLAMA model concern). In the future, we would love to have features like layer-level modifications.

Cookbook Usage We used the tinker-cookbook extensively, particularly the `rl.interleaved`, `rl.problem_env`, and `supervised.train` modules. We also used the cookbook as reference context when implementing custom components. However, the cookbook does not provide clear examples for integrating non-sandboxed local verifiers (e.g., subprocess-based execution) into GRPO pipelines. Existing examples target specific model and dataset combinations (e.g., Qwen + DeepCoder) and required substantial modification for our Llama setup.

Suggested Improvements Tinker would benefit from improved visibility into training behavior. These changes would reduce debugging time and improve usability. In particular:

- Provide explicit warnings when dataset exhaustion will limit training steps
- Report effective training steps alongside configured values
- Allow changes to more parts of the model and training process

We would use Tinker again! It provides an efficient interface for LoRA fine-tuning and evaluation. The managed training and inference workflow reduces engineering overhead and enables rapid experimentation. Also, its free compute credits were very nice!

Reproducibility

4.1 Supervised Fine-Tuning (SFT)

```
caffeinate -dimsu .venv/bin/python evaluation/train_and_publish.py \
  --base_model "meta-llama/Llama-3.1-8B" \
  --num_steps 6400 \
  --batch_size 10 \
  --lr 5e-5 \
  --num_tulu 15000 \
  --num_gsm8k 12000 \
  --num_opencode 5000 \
  --checkpoint_name "sft_8b_tulu15000_metamath12000_opencode5000_6400steps_bs10"
```

4.2 Direct Preference Optimization (DPO)

```
caffeinate -dimsu .venv/bin/python -m tinker_cookbook.recipes.preference.dpo.train \
  model_name="meta-llama/Llama-3.1-8B" \
  dataset="ultrafeedback" \
  load_checkpoint_path="<Saved state path of SFT model>." \
  learning_rate=1e-5 \
  dpo_beta=0.1 \
  batch_size=32 \
  max_steps=200 \
  behavior_if_log_dir_exists="delete" \
  log_path="/tmp/tinker-examples/dpo/8b_true_dpo_ultrafeedback_200_from_best_sft"
```

4.3 Group Relative Policy Optimization (GRPO)

```
caffeinate -dimsu .venv/bin/python -m tinker_cookbook.recipes.rl_loop \
  model_name="meta-llama/Llama-3.1-8B" \
  load_checkpoint_path="<State path of SFT model>" \
  batch_size=32 \
  group_size=8 \
  learning_rate=2e-5 \
  lora_rank=32 \
  max_tokens=256 \
  save_every=50 \
  log_path="/tmp/tinker-examples/grpo/8b_from_sft"
```

End-to-end time is around $\sim$5 hours